

A Complete, Certified Search for Stationary Markov Perfect Equilibria in Farsighted Coalition Formation

Strategy document — worked through the `burke_usaruschn.2035--2060` instance

June 20, 2026

Abstract

This document lays out, step by step, the strategy we intend to pursue for *certifiably* finding a stationary Markov Perfect Equilibrium (MPE) of the farsighted coalition-formation game, or characterising why doing so is hard. It is written to be judged: before committing weeks or months of compute, we want the mathematics and the combinatorial accounting to be transparent and correct. Every abstract object (player, state, transition, committee, weak order, “degeneracy”) is illustrated on one concrete instance, the payoff table `burke_usaruschn.2035--2060.xlsx`, which we have selected as the running example and first target. We are explicit about where a number is an exact count and where it is a (possibly loose) upper bound, because an earlier version of our own reasoning conflated the two.

Contents

1	The game instance	2
1.1	Players and the running example	2
1.2	States (coalition structures)	2
1.3	Timing, protocol, discounting	3
1.4	Which transitions are allowed: the effectivity rule <code>adjacent_step</code>	3
1.5	Who must approve: committees	3
2	Strategies	4
3	Value functions and the equilibrium conditions	4
3.1	Continuation values	4
3.2	The two rationality conditions	4
4	Reduction to ordinal rankings, and the 541³ count	5
4.1	Why only the ordering of V matters	5
4.2	Strict orders, weak orders, and the count	5
5	Degeneracy: definition and why this instance is hard	5
6	What must be done to test one weak-order triple	6
6.1	Step 1 — classify the acceptance decisions (instant)	6
6.2	Step 2 — the feasibility problem	6
6.3	Step 3 — solve that system, completely	7

7	Counting the work: components, interaction, and expected duration	8
7.1	The two components and how they combine	9
7.2	A worked example: a maximum-work and a minimum-work label	9
7.3	Component 1: the mixing dimension m and the solve cost $c(m)$	10
7.4	Component 2: the proposal-support count r	11
7.5	Interaction: the joint distribution of r and m	12
7.6	Putting it together: the expected duration	13
8	Scope of the guarantee	15
9	What this document commits to, and what remains open	16
A	Peripheral sub-analyses (not load-bearing)	16
A.1	A loose upper bound: “conceivable combinations”	16
A.2	Variable elimination and realisable supports	16
A.3	Summary of the key quantities	17

1 The game instance

1.1 Players and the running example

The game has $n = 3$ players. In the running instance they are the economic blocs **CHN**, **RUS**, **USA**. Each may participate in a coalition that deploys solar geoengineering (SG). The instance is defined by a payoff table; its contents are shown in Figure 1.

State	CHN	RUS	USA	W_SAI_sum_2035-2060
()	-129,563160	-12,604171	-23,225735	0,000
(CHN)	-128,319196	-13,790740	-23,138532	167,855
(RUS)	-129,563160	-12,604171	-23,225735	0,000
(USA)	-128,364223	-13,253407	-23,131380	102,558
(CHNRUS)	-128,887140	-12,921883	-23,121155	61,747
(CHNUSA)	-128,216505	-13,497373	-23,161321	137,333
(RUSUSA)	-129,443678	-12,641797	-23,214372	25,726
(CHNRUSUSA)	-128,749769	-12,963200	-23,144657	72,359

Figure 1: The payoff table `burke_usaruschn_2035--2060`. Each row is a coalition structure; the columns **CHN**, **RUS**, **USA** give the static payoff $u_i(x)$ to each player in that structure, and the last column is the implied SG deployment level G . Payoffs are (negative) economic outcomes, so larger (less negative) is better.

1.2 States (coalition structures)

A *state* is a coalition structure: who cooperates with whom. The table lists eight rows, but under the *power-threshold* governance rule a single country cannot deploy SG (it lacks the required power share, here $\geq 50.1\%$ with equal power $1/3$ each). Hence the three singleton rows (CHN), (RUS), (USA) are payoff-identical to the no-cooperation state () — indeed in Figure 1 the rows (), (RUS) and the deployment column all read $G = 0$. The framework therefore collapses to the $S = 5$ **canonical states**:

$$\mathcal{X} = \{ (), (CHNRUS), (CHNUSA), (RUSUSA), (CHNRUSUSA) \}.$$

Concretely (CHNRUS) is “CHN and RUS cooperate and jointly deploy; USA is alone,” and (CHNRUSUSA) is the grand coalition. The static payoff matrix $u_i(x)$ we actually use (extracted from the table) is

state x	$u_{\text{CHN}}(x)$	$u_{\text{RUS}}(x)$	$u_{\text{USA}}(x)$	$G(x)$
()	-129.563160	-12.604171	-23.225735	0.000
(CHNRUS)	-128.887140	-12.921883	-23.121155	61.747
(CHNUSA)	-128.216505	-13.497373	-23.161321	137.333
(RUSUSA)	-129.443678	-12.641797	-23.214372	25.726
(CHNRUSUSA)	-128.749769	-12.963200	-23.144657	72.359

Table 1: The five-state payoff matrix used by the solver, with deployment levels.

1.3 Timing, protocol, discounting

Time is discrete. In each period one player is drawn as *proposer* according to a fixed protocol ρ ; here it is uniform, $\rho_{\text{CHN}} = \rho_{\text{RUS}} = \rho_{\text{USA}} = \frac{1}{3}$. The chosen proposer may propose moving to a different state; an *approval committee* then votes; if it approves, the system moves, otherwise it stays. Future payoffs are discounted by $\delta = 0.99$.

1.4 Which transitions are allowed: the effectivity rule `adjacent_step`

Not every state-to-state move can be proposed. We use the effectivity rule `adjacent_step`: a proposer may only propose a move that changes the deploying coalition by *one membership event* (one country joins, or one leaves, or a pair forms/dissolves). Concretely, for the running instance the allowed (non-forbidden) moves form a “diamond”:

$$(\) \longleftrightarrow \{ (\text{CHNRUS}), (\text{CHNUSA}), (\text{RUSUSA}) \} \longleftrightarrow (\text{CHNRUSUSA}),$$

plus the self-loop “stay” at every state. That is:

- From () one may form any pair, e.g. $(\) \rightarrow (\text{CHNRUS})$, but **not** jump straight to the grand coalition ($(\) \rightarrow (\text{CHNRUSUSA})$ is *forbidden* — that would be two events).
- From a pair, e.g. (CHNRUS), one may dissolve to () or admit the third member to reach (CHNRUSUSA); one may **not** jump to another pair ($(\text{CHNRUS}) \rightarrow (\text{CHNUSA})$ is forbidden — RUS leaves *and* USA joins).
- From the grand coalition one member may leave ($(\text{CHNRUSUSA}) \rightarrow (\text{CHNRUS})$); all three dissolving at once ($(\text{CHNRUSUSA}) \rightarrow (\)$) is forbidden.

Counting allowed targets (including “stay”) per state gives

$$\# \text{feasible}(\) = 4, \quad \# \text{feasible}(\text{pair}) = 3 \text{ (each)}, \quad \# \text{feasible}(\text{CHNRUSUSA}) = 4.$$

We write $\mathcal{F}(i, x)$ for the set of states proposer i may propose from x ; under this rule it does not depend on i , so e.g. $\mathcal{F}(\cdot, (\)) = \{(\), (\text{CHNRUS}), (\text{CHNUSA}), (\text{RUSUSA})\}$. `adjacent_step` is deliberately restrictive: fewer feasible transitions means smaller approval committees and a smaller search, which is exactly why we prefer it over the looser 2021 rule.

1.5 Who must approve: committees

For each potential move “proposer i proposes $x \rightarrow y$ ” the governance structure fixes an *approval committee* $C(i, x, y) \subseteq \{\text{CHN}, \text{RUS}, \text{USA}\}$ whose consent is required (unanimously, since `unanimity_required` is true here). These are fixed rules, *not* decision variables. Examples from the running instance:

- CHN proposes $(\) \rightarrow (\text{CHNRUS})$: committee = {RUS} (CHN wants in; RUS must consent to pair with CHN; USA is unaffected).
- CHN proposes $(\) \rightarrow (\text{RUSUSA})$: committee = {RUS, USA} (CHN proposes a coalition it is *not* part of; both members must consent).
- CHN proposes $(\text{CHNRUSUSA}) \rightarrow (\text{RUSUSA})$ (CHN exits the grand coalition): committee = {CHN} — a country may leave unilaterally.

2 Strategies

A (stationary, Markov) strategy profile σ specifies two kinds of behaviour.

Proposal probabilities. For each proposer i and current state x , a probability distribution over feasible next states: $\sigma_i^P(x, \cdot)$ on $\mathcal{F}(i, x)$, summing to 1. Example: “when USA is drawn in state (CHNRUS), it proposes (CHNRUSUSA) with probability 1” is the pure proposal $\sigma_{\text{USA}}^P((\text{CHNRUS}), (\text{CHNRUSUSA})) = 1$.

Acceptance probabilities. For each committee member $k \in C(i, x, y)$, the probability $\sigma_k^A(x, y | i) \in [0, 1]$ that k votes to approve. Only committee members have a decision; others are undefined. Example: in the move CHN proposes $(\cdot) \rightarrow (\text{CHNRUS})$, the only decision is RUS’s acceptance probability.

We index an acceptance decision by the quadruple (i, x, y, k) : *proposer i , from x , to y , approver k* . We call (i, x, y, k) a *context*. The same underlying preference of k (does k like y better than x ?) can appear in several contexts (different proposers i), which will matter for variable elimination in Section 7.4.

3 Value functions and the equilibrium conditions

3.1 Continuation values

A profile σ induces a transition matrix $P = P(\sigma)$ on the five states: the probability of moving $x \rightarrow y$ in one period, obtained by summing over who is drawn (ρ_i), what they propose (σ^P), and whether the committee approves ($\prod_{k \in C} \sigma_k^A$, by unanimity). Given P , the long-run value to player i in state x solves the Bellman equation

$$V_i(x) = (1 - \delta) u_i(x) + \delta \sum_y P(x, y) V_i(y), \quad \text{i.e.} \quad V_i = (I - \delta P)^{-1} (1 - \delta) u_i. \quad (1)$$

Crucially, $P(\sigma)$ is a *multilinear* (hence polynomial) function of the strategy probabilities, and V is therefore a *rational* function of σ (ratios of determinants). This polynomial structure is what later makes exact, certified solving possible in principle.

3.2 The two rationality conditions

A profile is a stationary MPE iff, simultaneously:

(A) Approval rationality. A committee member approves iff the destination improves its own value:

$$\sigma_k^A(x, y | i) = \begin{cases} 1 & \text{if } V_k(y) > V_k(x), \\ 0 & \text{if } V_k(y) < V_k(x), \\ \text{any value in } [0, 1] & \text{if } V_k(y) = V_k(x). \end{cases}$$

(P) Proposal rationality. A proposer puts positive probability only on moves that maximise its expected continuation value, accounting for the chance of approval:

$$\sigma_i^P(x, y) > 0 \implies y \in \arg \max_{y' \in \mathcal{F}(i, x)} \left[p^A(i, x, y') V_i(y') + (1 - p^A(i, x, y')) V_i(x) \right],$$

where $p^A(i, x, y') = \prod_{k \in C(i, x, y')} \sigma_k^A(x, y' | i)$ is the committee approval probability and the bracket is the *expected value* EV of proposing y' .

The difficulty is the coupling: σ determines P , which determines V via (1), which determines what is rational in (A) and (P), which determines σ . An equilibrium is a fixed point of this loop. Existence is not in doubt: Fink’s theorem guarantees that every finite discounted stochastic game has at least one stationary equilibrium in behavioural (mixed) strategies. The problem is *finding* one, with certainty.

4 Reduction to ordinal rankings, and the 541^3 count

4.1 Why only the ordering of V matters

Inspect conditions (A) and (P): both depend on V *only through comparisons*. Condition (A) asks whether $V_k(y)$ is greater than, less than, or equal to $V_k(x)$. Condition (P) asks which y' *maximises* an expression; away from ties, the maximiser is determined by the order of the relevant values. Therefore the *qualitative* equilibrium behaviour is fixed once we know, for each player, the **ordinal ranking** of the five continuation values $V_k(\cdot), \dots, V_k(\text{CHNRUSUSA})$.

4.2 Strict orders, weak orders, and the count

For one player there are $S = 5$ states. If all five values are distinct, the ranking is a *strict total order* — a permutation — and there are $5! = 120$ of them. But values can be *equal* (Section 5 explains why this is the generic, not exceptional, case here), and equalities are exactly where mixing lives. A ranking that allows ties is a *weak order* (a ranking with possible “tied groups”). The number of weak orders on S items is the *ordered Bell number* (Fubini number) $a(S)$:

$$a(5) = 541.$$

(For intuition: $a(5)$ counts the ways to partition 5 states into an ordered sequence of non-empty tied groups; e.g. the partition giving rank $(\text{CHNUSA}) \succ \{(\cdot) = (\text{RUSUSA})\} \succ \{(\text{CHNRUS}) = (\text{CHNRUSUSA})\}$ is one weak order for a player with those three preference tiers.)

Because the three players’ rankings are independent objects, a complete enumeration of the qualitative structure is a **triple** of weak orders — one per player. Hence the number of *weak-order triples* (we also call these *labels*) is

$$a(5)^3 = 541^3 = 158,340,421 \approx 1.6 \times 10^8. \quad (2)$$

This $\approx 1.6 \times 10^8$ is the size of the outer search: every qualitative equilibrium structure is one of these 541^3 triples. (The strict-only count is $120^3 = 1,728,000$; the gap between 120^3 and 541^3 is precisely the ranking structures that involve ties — the mixed-strategy candidates.)

Remark 1. *The number sometimes quoted as “ 543^3 ” is a rounding; the exact Fubini number is $a(5) = 541$.*

5 Degeneracy: definition and why this instance is hard

Definition 1 (Ordinal cell). *Fix a weak-order triple R . The set of value matrices $V \in \mathbb{R}^{S \times n}$ whose per-player orderings (including which states are tied) match R is a region of value space; we call it the ordinal cell of R . Within a cell, conditions (A) and (P) prescribe the same qualitative behaviour.*

An equilibrium “lives in” the ordinal cell of its own value ordering. The cell’s size in any player’s row is governed by the *gaps* between that player’s distinct continuation values: if two states are nearly tied, the cell is a thin sliver in that direction.

Definition 2 (Degeneracy). We call an instance degenerate to the extent that the payoffs — and hence the induced continuation values — give nearly identical numbers to a player across different states. Quantitatively we use the smallest gap between a player’s sorted state-payoffs, normalised by that player’s payoff range,

$$\deg_i = \frac{\min_{x \neq x'} |u_i(x) - u_i(x')|}{\max_x u_i(x) - \min_x u_i(x)}, \quad \deg = \min_i \deg_i,$$

and we say the instance is more degenerate when $\deg$ is smaller.

Degeneracy has two consequences, both bad for naive search:

1. **Tiny ordinal cells.** Small value gaps make the equilibrium’s cell a needle in $\mathbb{R}^{S \times n}$. Random or gradient search in value space rarely lands in it. (We measured this on related instances: perturbing an equilibrium’s value matrix by less than the smallest gap keeps the induced strategy unchanged; perturbing by more flips it.)
2. **Exact ties $\Rightarrow$ mixing.** When the equilibrium genuinely sets two of a player’s values equal, that player is *indifferent*, condition (A) permits any acceptance probability, and the equilibrium is *mixed*. The precise mixing probability is not free: it is pinned by the requirement that the indifference actually holds (Section 6).

For the running instance, using static payoffs as a proxy, $\deg \approx 4.2 \times 10^{-2}$ (the binding player/pair being USA, whose values across states differ by only a few percent of its range). Among the unsolved tables this is the *least* degenerate — by contrast another instance, `burke_rusndechn`, has $\deg \approx 2 \times 10^{-6}$ (essentially flat, numerically hopeless). This is exactly why we chose `burke_usaruschn` as the first target: it is hard enough to be unsolved by every existing method, yet the least degenerate, so its equilibrium cells are as large as we can hope for and its mixing dimension as small as possible.

6 What must be done to test one weak-order triple

We now describe the work for a *single* label $R = (R_{\text{CHN}}, R_{\text{RUS}}, R_{\text{USA}})$. This is the heart of the algorithm; the outer loop simply runs it for each of the 541³ labels.

6.1 Step 1 — classify the acceptance decisions (instant)

For each context (i, x, y, k) with $k \in C(i, x, y)$, the label fixes k ’s ranking of x and y , so condition (A) gives one of three modes:

$$\begin{cases} \text{strict accept } (\sigma_k^A = 1) & \text{if } R_k \text{ ranks } y \succ x, \\ \text{strict reject } (\sigma_k^A = 0) & \text{if } R_k \text{ ranks } y \prec x, \\ \text{free / mixing } (\sigma_k^A = a \in [0, 1]) & \text{if } R_k \text{ ranks } y \sim x. \end{cases}$$

Example. Suppose R_{RUS} ranks $(\text{CHNRUS}) \succ ()$. Then in “CHN proposes $() \rightarrow (\text{CHNRUS})$ ” (committee $\{\text{RUS}\}$), RUS strictly accepts: that edge is on with probability 1. If instead R_{RUS} ranks $(\text{CHNRUS}) \sim ()$, RUS is indifferent and its acceptance becomes a free variable a — a mixing unknown.

The *free* acceptances are the only acceptance unknowns; strict ones are constants. Their number is small (it equals the count of indifferent committee contexts), and many can be eliminated immediately if they sit on transitions that no one ever proposes (Section 7.3).

6.2 Step 2 — the feasibility problem

The unknowns of the label are:

- the free acceptance probabilities $a_1, \dots, a_m \in [0, 1]$ (the indifferent contexts of Step 1), and
- the proposal probabilities $\sigma_i^P(x, \cdot)$ on $\mathcal{F}(i, x)$ for each (i, x) — distributions over the feasible targets.

Given these, P is determined, and V is determined from (1). The label corresponds to an equilibrium iff there exist values of these unknowns such that all of the following hold:

- (i) **Ordinal consistency.** The induced V reproduces R : every strict ranking $y \succ x$ of R_k becomes the strict inequality $V_k(y) > V_k(x)$, and every tie $y \sim x$ becomes the *equation* $V_k(y) = V_k(x)$. The equations are the *indifference conditions*; they are what pin the mixing probabilities.
- (ii) **Proposal optimality.** Each proposer puts mass only on EV-maximising feasible targets (condition (P)).
- (iii) **Box constraints.** All probabilities lie in $[0, 1]$ (and proposal rows sum to 1).

Because V is rational in the unknowns, clearing denominators turns the indifference equations into *polynomial* equations, and (P) into polynomial *complementarity* conditions ($\sigma_i^P(x, y) \cdot (\text{EV}_{\max} - \text{EV}(y)) = 0$, with both factors sign-constrained). So testing a label is exactly: *decide whether a system of polynomial equations and inequalities has a solution in a box, and if so produce one.*

6.3 Step 3 — solve that system, completely

What “feasibility” and a “feasible point” mean. Step 2 produced a *system of polynomial equations and inequalities* in the label’s unknowns (the free acceptance probabilities and the proposal probabilities). We are *not* optimising anything — there is no objective function to maximise. We are asking a pure existence question:

Feasibility: does there exist an assignment of numerical values to all the unknowns that satisfies *every* constraint (i)–(iii) simultaneously?

Such an assignment is called a *feasible point*. A feasible point is precisely a concrete equilibrium strategy for this label: specific mixing probabilities and proposal probabilities whose induced value matrix V makes all the indifference *equations* hold exactly, respects all the strict-order *inequalities*, has the proposers optimising, and keeps every probability in $[0, 1]$. A “semi-algebraic feasibility problem” is just this question for constraints that are polynomial equalities and inequalities (“semi-algebraic” = defined by polynomial signs).

Example. Take a label in which RUS is indifferent between (CHNRUS) and ($\emptyset$), so RUS’s acceptance of CHN’s move ($\emptyset$) $\rightarrow$ (CHNRUS) is a free unknown $a \in [0, 1]$. A feasible point is a specific number a^* such that the value matrix it induces actually makes RUS indifferent there ($V_{\text{RUS}}((\text{CHNRUS})) = V_{\text{RUS}}(\emptyset)$), while every strict order required by the label holds and every proposer is optimising. If such an a^* exists, the label yields an equilibrium; if no value in $[0, 1]$ does, the label is infeasible and contributes no equilibrium. The whole search is: scan the 541³ labels, decide feasibility of each, and return a feasible point of any label that has one.

Why “complete”. The system is decidable exactly (Tarski–Seidenberg), and in practice solvable by certified methods — exact Gröbner bases with real-root isolation, cylindrical algebraic decomposition, or certified homotopy continuation. “Complete” means: if a feasible point exists the method is guaranteed to find one, and it does not silently miss solutions. This is the property that least-squares/fixed-point heuristics lack (they can converge to nothing even when a solution exists), and it is what makes the search *waterproof within its regime*.

Zero- vs. positive-dimensional solution sets (continua of equilibria). The set of *all* feasible points of a label is some subset of the unit box. Its *dimension* is the number of independent directions in which one can move and *stay* a solution:

- **Zero-dimensional:** the feasible points are *isolated* — finitely many separated equilibria. (Generic when the number of independent equations equals the number of unknowns: the equations pin every variable.)
- **Positive-dimensional:** the feasible set contains a continuous piece — a curve (dimension 1), surface (dimension 2), ... — so there are *infinitely many* equilibria forming a *continuum*. (Generic when there are *fewer* independent equations than unknowns: the system is underdetermined and leaves free directions.)

Terminology (one concept, two names). “Positive-dimensional solution set” and “ d -parameter family of equilibria” denote the *same* thing: a solution set of dimension d is exactly a family parametrised by d freely-varying quantities, i.e. dimension = number of free parameters. So “positive-dimensional” is the general statement ($d \geq 1$, a continuum), and the “one-parameter family” discussed below is simply the case $d = 1$ (a curve). Throughout we reserve the words *family* and *continuum* for this positive-dimensional case ($d \geq 1$); we never use them for a finite collection of isolated ($d = 0$) equilibria.

Positive-dimensional solution sets are not a pathology to be avoided — under the heavy degeneracy of these instances they are exactly what an equilibrium often looks like. They are also precisely what numerical fixed-point and least-squares methods mishandle (those methods chase *a* point and have no notion of a solution manifold). A complete solver, by contrast, detects and represents them (e.g. via real-root isolation of the reduced system, or numerical irreducible decomposition).

Why the solution can be a *whole family* of equilibria. The reason a continuum appears is the count of equations versus unknowns from Section 7.3: the indifference *equations* are few (at most $S - 1 = 4$ per player), but a single indifference of player k between two states can be realised through *several distinct acceptance contexts* (the same preference appears for different proposers), so there can be *more mixing unknowns than indifference equations*. When that happens the equations carve out a solution set of dimension (unknowns – independent equations) > 0 : a family.

Concrete numbers (from the reduced validation game). In the small game we used to test the solver, one player’s single indifference produced two mixing unknowns a_1, a_2 tied by *one* linear indifference equation, which the exact solver returned as

$$a_1 = 3a_2 + \frac{2}{33}.$$

This is one equation in two unknowns, so its solution set is a *line*: a_2 is a *free parameter*, and for every a_2 in the interval that keeps both $a_1, a_2 \in [0, 1]$ (here $a_2 \in [0, (1 - \frac{2}{33})/3]$) the pair (a_1, a_2) is a genuine, independently verifiable equilibrium. Hence a **one-parameter family** of equilibria — a line segment of them — rather than a unique point. Intuitively: a player who is exactly indifferent has *no unique* rational mixing level; a range of mixing levels all keep the indifference self-consistent, so the equilibrium is not pinned to a single value but spread over a continuum. (This was a correctness test on a *known* reduced game — the solver recovered the known continuum and an independent verifier confirmed points on it — not a new solve of an unsolved table.)

7 Counting the work: components, interaction, and expected duration

This section is organised as a map. First we name the *two cost components* and the single formula by which they combine into total runtime. Then we analyse each component on its own, then their *interaction* (their joint distribution), and finally we read off what it all means for the expected duration of a complete — and near-complete — sweep. All numbers are for the running instance `burke_usaruschn`; the measurement code is in `write-up/measurements/`.

7.1 The two components and how they combine

The outer loop is fixed and unambiguous: the $a(5)^3 = 541^3 \approx 1.6 \times 10^8$ weak-order triples (“labels”) of Section 4. The cost of a complete search is the sum over labels of a per-label cost,

$$\text{total} = \sum_R C(R),$$

and the per-label cost factors into the two components:

$$C_{\text{full}}(R) \approx \underbrace{r(R)}_{\substack{\# \text{ candidate} \\ \text{proposal supports}}} \times \underbrace{c(m(R))}_{\substack{\text{cost of one} \\ \text{exact solve}}}, \quad C_{\text{acc}}(R) = c(m(R)) \quad (\text{proposals fixed, } r=1). \quad (3)$$

Here $m(R)$ is the *acceptance mixing dimension* (how many acceptance variables are free, Section 6.1); $c(m)$ is the cost of *one* exact per-label solve at that dimension; and $r(R)$ is the number of candidate proposal supports a *complete* full search must consider for that label (the acceptance-class search fixes proposals, so $r=1$). Equation (3) for C_{full} is a *lower-bound* model: a support with mixed proposals adds variables, so its solve costs at least $c(m)$.

The crucial structural point, developed below, is that *both* components are produced by the same feature — **ties in the weak order** R — so they are *correlated*, and the cost piles up on the same small set of tie-rich labels. The roadmap is therefore: §7.3 analyses m and $c(m)$; §7.4 analyses r ; §7.5 analyses their joint distribution (the interaction); §7.6 gives the totals.

7.2 A worked example: a maximum-work and a minimum-work label

Before the distributions, it helps to see *why* one label can cost a fraction of a millisecond and another can cost geological time — from the *same* 15-cell skeleton, differing only in how many *ties* each player’s ranking has. Recall $r(R) = \prod_{\text{cells}} (2^t - 1)$ with t the number of feasible targets at the proposer’s best tier, and $m(R)$ the number of indifferent acceptance contexts.

Minimum-work label: every player strictly orders the five states. Take the ranking $() \succ (\text{CHNRUS}) \succ (\text{CHNUSA}) \succ (\text{RUSUSA}) \succ (\text{CHNRUSUSA})$ for all three players (all values distinct, no ties).

- *Proposals.* In every (proposer, state) cell the proposer has a *unique* best feasible target ($t = 1$), so each factor is $2^1 - 1 = 1$ and

$$r = \underbrace{1 \cdot 1 \cdots 1}_{15 \text{ cells}} = 1 :$$

every proposal is pure, there is nothing to enumerate.

- *Acceptances.* No approver is ever indifferent (all values distinct), so every acceptance is forced to 0 or 1: $m = 0$, no mixing variables.
- *Cost.* One exact linear solve for V plus inequality checks: $C = c(0) \approx \mathbf{0.0001 \text{ s}}$. Instant.

Maximum-work label: every player indifferent among all five states. Now take the fully-tied ranking $() = (\text{CHNRUS}) = (\text{CHNUSA}) = (\text{RUSUSA}) = (\text{CHNRUSUSA})$ for all three players (one tier, everything equal).

- *Proposals.* In every cell the proposer is indifferent among *all* its feasible targets, so the best tier contains all of them: $t = |\mathcal{F}(i, x)|$. With the feasibility counts $(4, 3, 3, 3, 4)$ the per-cell factors are $(15, 7, 7, 7, 15)$, so per proposer $15 \cdot 7 \cdot 7 \cdot 7 \cdot 15 = 77,175$, and across the three proposers

$$r = 77,175^3 = \mathbf{459,652,804,734,375} \approx 4.6 \times 10^{14}$$

candidate proposal supports — the global maximum of r over all 541^3 labels.

- *Acceptances.* Every approver is indifferent on every transition; after eliminating the irrelevant ones, $m = 9$ acceptance variables remain, with $c(9) \approx 25$ s per solve.
- *Cost.* $C_{\text{full}} \approx r \cdot c(m) = 4.6 \times 10^{14} \times 25 \text{ s} \approx 1.2 \times 10^{16} \text{ s} \approx \mathbf{3.7 \times 10^8 \text{ years}}$ — for this *one* label. Even at an optimistic 1 ms per support, 4.6×10^{14} supports is still $\approx \mathbf{14,600 \text{ years}}$.

The lesson. The two labels share the identical state set, adjacency, and committees; they differ *only* in the tie-structure of the three rankings, yet their costs differ by a factor of $\sim 10^{19}$. Ties are the sole source of both components: each tie at a proposer’s top widens a support factor ($t \uparrow, 2^t - 1 \uparrow$), and each indifferent approver adds a mixing variable ($m \uparrow, c(m) \uparrow$). A *typical* triple sits near the strict end (median $m = 2$, $r = 27$: a couple of ties, cost ~ 0.002 s); a vanishing minority sits near the fully-tied extreme. This is the entire reason the cost is so concentrated, and why the analyses below separate the cheap bulk from the tie-rich tail.

7.3 Component 1: the mixing dimension m and the solve cost $c(m)$

What m is, and its distribution. $m(R)$ counts the indifferent acceptance contexts that survive on proposed transitions (Section 6.1). It is small: over 2×10^5 random triples, median $m = 2$, mean 1.9, 99th percentile 7, maximum 14; 23% of triples have $m = 0$ (pure, no acceptance mixing at all).

What $c(m)$ is. $c(m)$ has two parts, *build* (forming the exact indifference polynomials) and *solve* (finding their roots). Profiling (`bench_exact_methods.py`) showed the build originally dominated — the symbolic matrix inverse $V = (I - \delta P)^{-1}(1 - \delta)u$ blew up with m (seconds, with > 30 s outliers). We replaced it with an **exact interpolation build**: each mixing variable has degree ≤ 1 in P , so $\det(I - \delta P)$ and the numerators are *multilinear*, hence determined by their values on the 2^m vertices $\{0, 1\}^m$; we evaluate those exactly with FLINT rational solves (`fmpq_mat`) and recover the polynomials by a fast Möbius transform. This is fully exact (no tolerance) and **25–640×** faster than the symbolic build; it was validated end-to-end (it reproduces the manifold game’s exact system and a verified equilibrium). With the build thus made negligible (≤ 1.4 s even at $m = 11$), the *solve* is now what grows with m :

m	≤ 2	3	4	5	6	7	8
$c(m)$ [s]	≤ 0.002	0.011	0.044	0.63	0.65	9.9	18.6 (+timeouts)

i.e. $c(m)$ is *super-linear* (geometric, roughly $\times 5$ per unit m), and past $m \approx 8$ the exact `sympy` solve on the (expanded, up to 2^m -term) multilinear system hits our measurement cap. This is the one place a faster exact multivariate engine (e.g. `msolve`) would help.

Where the time went (and the exact fix). Profiling split $c(m)$ into *build* (forming the polynomials) and *solve* (rooting them). The build originally dominated:

m	symbolic build (s)	solve (s)
1–3	0.03–1.0	≤ 0.002
4–5	0.1–7.2	≤ 0.32
6–8	0.4–>30 (DNF)	≤ 0.47

The blow-up was `sympy`’s symbolic inverse $V = (I - \delta P)^{-1}(1 - \delta)u$. The exact interpolation build (multilinearity $\Rightarrow 2^m$ FLINT vertex solves + Möbius) removed it (25–640× faster, validated), after which the build is ≤ 1.4 s even at $m = 11$ and the residual cost is the solve. The exact methods used, by case (each unfailable):

case	build	solve
all m (the bottleneck)	FLINT <code>fmpq_mat</code> interpolation	—
$m = 0$ (pure)	one exact rational solve	inequality check (exact)
$m = 1$	—	FLINT <code>fmpq_poly</code> /Arb or <code>sympy</code> real-roots (certified)
$m \geq 2$	—	<code>sympy</code> Gröbner/solve (exact)

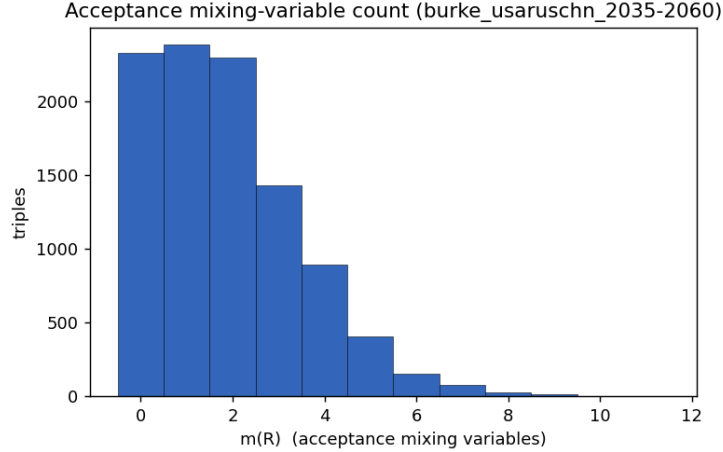


Figure 2: Distribution of the acceptance mixing dimension $m(R)$ over 10^4 random triples (median 2, max 14; 23% have $m = 0$). This is the cheap a-priori predictor of per-label cost: solve time rises super-linearly in m .

7.4 Component 2: the proposal-support count r

What r is. For one (proposer, state) cell the *proposal support* is the set of feasible targets that receive positive probability (a singleton = pure proposal; size ≥ 2 = the proposer mixes). The proposer can only mix over targets that are *co-maximal* in expected value, which generically means targets it ranks *tied* at the top. Counting non-empty subsets of the top-tier-tied feasible targets and multiplying over the 15 cells gives

$$r(R) = \prod_{\text{cells}} (2^t - 1), \quad t = \#\{\text{feasible targets at the proposer's best tier}\}.$$

With the feasibility counts (4, 3, 3, 3, 4) per proposer (the adjacency “diamond” of Section 1.4), a cell with no top-tie has $t=1$ and contributes a factor 1 (pure, nothing to enumerate); a cell with a k -way top-tie contributes $2^k - 1$. So r is 1 exactly when every proposer has a strict best move, and grows only where R supplies ties.

Distribution. Over 2×10^5 triples, r is **extremely heavy-tailed**: median 27, $p_{90} = 2,835$, $p_{99} = 5.8 \times 10^5$, $p_{99.9} = 3.7 \times 10^7$, mean 1.3×10^6 , maximum 1.2×10^{11} . The mean sits far above the median and grows with sample size — the signature of a distribution whose total is dominated by its largest few values.

Caveat (it is itself an upper bound on realisable supports). $r(R)$ counts *candidate* supports; a support is truly realisable only if its targets can be made exactly EV-tied, an extra equation most candidates fail. So r over-counts the realisable supports. But for a *complete* (waterproof) search this does not reduce the work: one cannot skip a candidate without testing

it, so all $r(R)$ candidates must be handled regardless. (The often-quoted 4.6×10^{14} all-subsets-of-all-feasible figure is a still looser bound and is not the work.)

The feasible-target vector $(4, 3, 3, 3, 4)$, and why it is structural. How many targets each cell has is the per-proposer vector $(|\mathcal{F}(i, ())|, \dots, |\mathcal{F}(i, (\text{CHNRUSUSA}))|) = (4, 3, 3, 3, 4)$ in state order $((), (\text{CHNRUS}), (\text{CHNUSA}), (\text{RUSUSA}), (\text{CHNRUSUSA}))$ — read straight off the adjacency diamond of Section 1.4 (4 from $()$ and from the grand coalition, 3 from each pair).

Remark 2 (This vector is structural, not instance-specific). $(4, 3, 3, 3, 4)$ comes only from the state set and *adjacent_step*, not from the payoffs of Figure 1. Hence: (a) it is identical for all three proposers; (b) it is identical for every one of the 541^3 triples (a triple fixes value rankings, not feasible sets); and (c) it is identical for every $n = 3$ power-threshold *adjacent_step* table — verified directly on *burke_usaruschn*, *kalkuhl_usachnnde*, *andreoni_usachnnde* (all give $(4, 3, 3, 3, 4)$). Only the payoffs, hence the equilibria, differ; the feasibility skeleton is shared.

The number of candidate supports per cell is $2^{|\mathcal{F}|} - 1$ (each feasible target independently in/out of the support gives $2^{|\mathcal{F}|}$ subsets; subtract the empty one, since a proposer must always propose *something*, at minimum “stay”). So $(4, 3, 3, 3, 4) \mapsto (15, 7, 7, 7, 15)$; e.g. USA in (CHNRUS) has the 7 non-empty subsets of $\{(\), (\text{CHNRUS}), (\text{CHNRUSUSA})\}$ (three pure, three two-way mixes, one three-way mix).

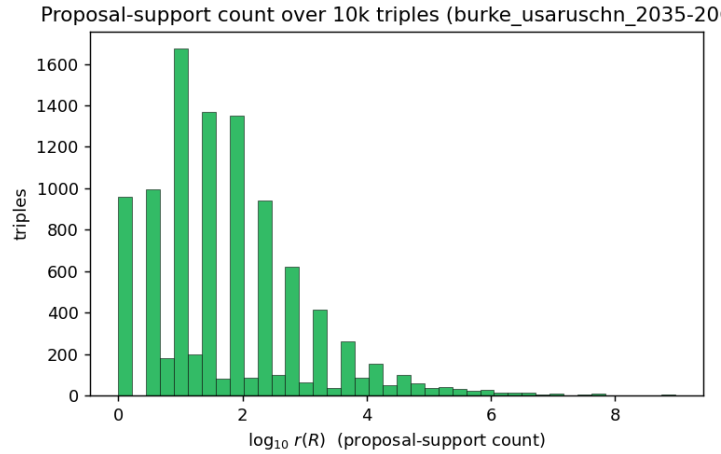


Figure 3: Distribution of $\log_{10} r(R)$ over 10^4 triples. Heavy-tailed: most labels are tiny (9.6% have $r = 1$, median 27) but the tail reaches $r \sim 10^{11}$, which is what makes the full search’s total ill-conditioned (§7.5).

7.5 Interaction: the joint distribution of r and m

The two components are not independent — both are manufactured by ties in R , so a tie-rich triple tends to have *both* a high m and a high r . Measured (*joint_analysis.py*, $N = 2 \times 10^5$):

- rank correlation $\rho_{\text{Spearman}}(r, m) = 0.44$ (clear positive coupling);
- median r rises steeply with m : 9 at $m=0$, 27 at $m=2$, 567 at $m=5$, 15,309 at $m=8$, 583,443 at $m=10$ (Figure 4).

Because the expensive-in- r labels are also the expensive-in- $c(m)$ labels, the *product* $r \cdot c(m)$ concentrates ferociously. Sorting labels by cost:

share of labels	% of <i>acceptance</i> cost ($c(m)$)	% of <i>full</i> cost ($r c(m)$)
top 1%	74.0%	99.998%
top 0.1%	18.7%	99.82%
top 0.01%	2.9%	97.4%
single most expensive (sampled) label	0.14%	79.2%

The full-cost column answers the natural question “does the top 1% really carry 100%, and the rest 0%?” — not literally: the top 1% carry 99.998% and the remaining 99% carry the small but non-zero remainder, 0.0024% (these are real labels with real, modest cost). What is true is the extreme concentration: a *single sampled* label carries 79% of the sample’s full cost. And the genuine worst label is far beyond any sample — it is the fully-tied triple of Section 7.2, with $r = 4.6 \times 10^{14}$, whose cost alone is $\sim 3.7 \times 10^8$ years. The acceptance cost is also concentrated (top 1% = 74%) but far less pathologically.

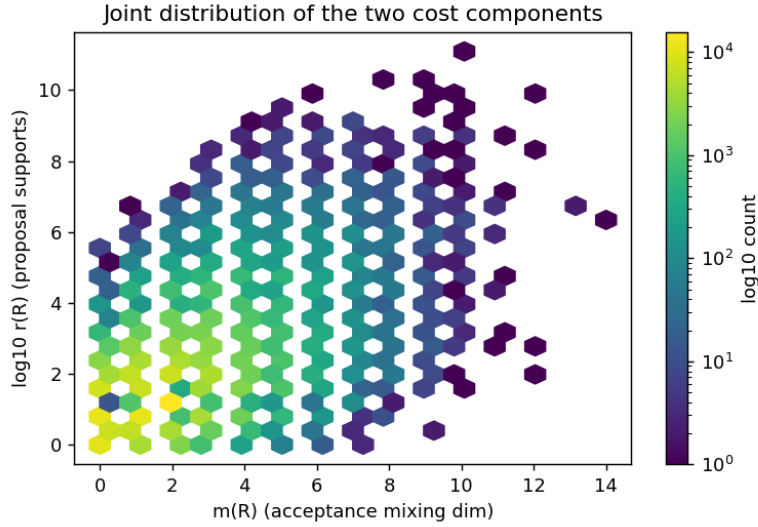


Figure 4: Joint distribution of the two cost components ($\log_{10} r$ vs. m) over 2×10^5 random triples. They are positively coupled: tie-rich triples sit in the upper-right (both large), and that corner — a vanishing fraction of labels — carries essentially all of the full-search cost.

7.6 Putting it together: the expected duration

Acceptance-class search: known with high confidence (direct measurement). With $r=1$ the cost is $\sum_R c(m(R))$, and this we can state with confidence for two independent reasons. First, the cost is *statistically stable*: $\mathbb{E}[c] = 0.21$ s with only 6.5% chunk-to-chunk scatter (§7.5). Second — and most faithfully — we did not model it but **ran the actual exact solver on a random sample of 10^4 labels** and extrapolated (`direct_sweep_timing.py`, 1,891 s wall, per-label mean 0.19 s, median 0.0015 s):

sweep (cheapest-first)	core-hours	on 14 workers	on 134 workers
cheapest 50% of labels	12	0.9 h	0.1 h
cheapest 90% of labels	323	23 h	2.4 h
cheapest 99% of labels	2,356	7.0 days	0.7 days
cheapest 99.9% of labels	7,435	22.1 days	2.3 days
complete (100%)	8,315	24.7 days	2.6 days

So the acceptance-class certified search for `burke_usaruschn` is ≈ 25 **days on 14 workers** (≈ 2.6 days on the 134-core farm), measured directly and consistent with the component estimate (~ 27 days) of Section 7.6’s category table. The complete and 99.9% rows are mild *lower* bounds (47 of the 10^4 labels hit a 20 s measurement cap), which a faster exact high- m solver removes; the cheapest 99% never touch the cap and are exact. Cheapest-first, predicting m for free, 90% of the entire search is done in under a day on 14 workers and the cost sits in the small tail.

Full proposal-mixing search: what we *can* estimate, and why the complete total is still out of reach. An earlier draft called the full cost “unmeasurable” — that was wrong, and worth correcting precisely, because both factors of $C_{\text{full}}(R) = r(R) c(m(R))$ are obtainable from cheaply-gathered data: $r(R)$ is a *deterministic structural count* (no solving), and $c(m)$ is *measured*. Consequently:

- **Any single label’s full cost is computable** — including the worst one. The most expensive label is not a mystery to be sampled: it is the fully-tied triple (Section 7.2), with $r = 4.6 \times 10^{14}$ and $c(m) \approx 25$ s, i.e. $\sim 10^{16}$ s $\approx 3.7 \times 10^8$ years *for that one label*. (One can also estimate any label’s c_{support} directly by solving a handful of its supports and multiplying by its known r — so even “sampling within a label” works; nothing is fundamentally unknowable pre-computation.)
- **The cheapest-fraction costs are computable and well-behaved** (table below): for the bulk of labels r and c are modest and well-sampled.
- **Only the *complete* total is statistically awkward to estimate by random sampling**, because it is dominated by the extreme tie-rich tail (in a 10^5 sample a single label already holds 79% of the cost; ten-way chunk means scatter 223% vs. 6.5% for acceptance). But we do not need a precise total: the *single* fully-tied label already costs $\sim 3.7 \times 10^8$ years, so the complete full search is infeasible by brute force regardless — a *computed* conclusion, not an unmeasured one.

Full-model budget table: “what share of labels fits in a given budget?” Because per-label full costs are computable, we can answer the planning question directly. Ordering labels cheapest-first by $r(R) c(m(R))$ and extrapolating to all 541³:

cheapest share of labels (full model)	core-hours	on 14 workers	on 134 workers
50%	231	0.7 days	0.07 days
90%	1.0×10^5	310 days	32 days
99%	4.2×10^7	1.2×10^5 days	1.3×10^4 days
99.9%	3.2×10^9	9.4×10^6 days	9.8×10^5 days

This is the full-model analogue of the acceptance table above (Figure 5 plots both concentration curves). The message: a *partial* full search is genuinely feasible up to a point — the cheapest $\sim 50\%$ of full-mixing labels finish in under a day on 14 workers, and one could push perhaps to the cheapest $\sim 70\text{--}80\%$ in a week or two — but the curve then turns sharply vertical (cheapest 90% already ~ 10 months on 14 workers), and the last fraction is dominated by the tie-rich tail and the single fully-tied label. So a budget buys a well-defined *share* of the full model, but never the complete sweep.

What this means. The two objects are sharply different. The **acceptance-class** search is a real, runnable, $\sim$ month-scale (14-worker) job whose time we know to $\sim 10\%$. The **full** search admits a feasible *partial* run (cheapest tens of percent) but never a complete one; its cost is concentrated on $< 0.01\%$ of labels — a handful of tie-rich orderings led by the fully-tied triple — so full feasibility reduces to whether that tail can be tamed: a *structural* reduction proving the extreme-tie labels cannot host (or cannot need to enumerate) so many supports. That is a mathematical question (Section 8), not a matter of more cores.

Acceptance-class time by case category. Binning all 541³ labels by m and weighting by the measured frequency (`measure_total_time.py`):

category	% of labels	core-hours	on 14 workers	on 134 workers
easy ($m \leq 2$)	69.4%	28	2.0 h	0.2 h
medium ($m=3-5$)	27.7%	1,378	98 h	10 h
hard ($m=6-8$)	2.85%	5,907	422 h	44 h
extreme ($m \geq 9$)	0.13%	1,739	124 h	13 h
total	100%	9,052	~ 27 days	~ 2.8 days

The easy 69% finish in ~ 2 hours; $> 85\%$ of the time is in the hard/extreme tail ($m \geq 6$, $< 3\%$ of labels). Figure 5 shows the cumulative cost-concentration (“Lorenz”) curves for both objects.

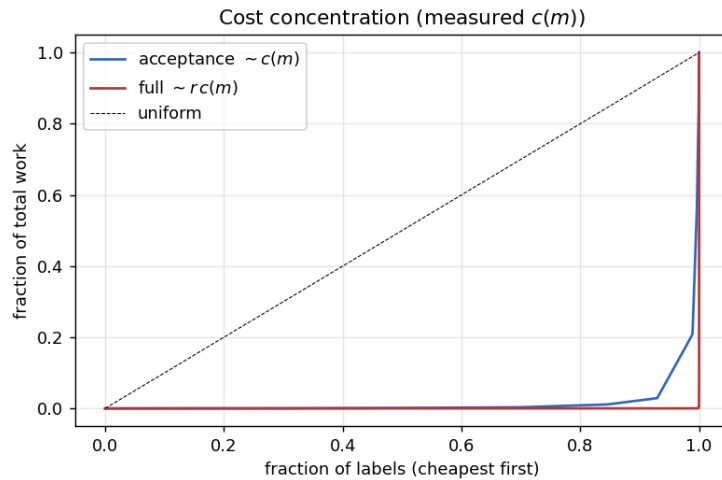


Figure 5: Cost concentration, cheapest-first, under the measured $c(m)$. x : fraction of labels processed; y : fraction of total work cleared. The acceptance curve (blue) is concentrated but workable; the full curve (red) hugs the corner — a single label holds 73% of its cost (§7.5).

8 Scope of the guarantee

We distinguish three regimes, because they have very different costs and the distinction has repeatedly tripped us up:

Acceptance mixing, fixed proposals. Enumerate 541³ labels; per label the proposal is pinned by a deterministic rule — concretely, each proposer is taken to propose its best-ranked *approvable* target (e.g. if USA in (CHNRUS) ranks (CHNRUSUSA) top among approvable moves, it proposes (CHNRUSUSA) purely) — and only acceptances mix. Certified and feasible with a fast backend. *Limitation:* if the equilibrium needs the proposer itself to mix (a genuinely two-element proposal support), this regime cannot represent it.

Full mixing (acceptances *and* proposals). The regime “that truly matters.” Per label the complementarity solve must also resolve proposal mixing. Certified; feasibility hinges on $r \cdot c$, whose joint distribution and totals are analysed in Section 7.5–7.6.

Find-one, not certified. Methods such as perturbation/homotopy continuation that return a full-mixing equilibrium without a completeness guarantee. Useful as an independent cross-check and as a fast route to a first solution.

9 What this document commits to, and what remains open

The strategy is: *enumerate the 541^3 weak-order triples; for each, eliminate irrelevant variables, build the exact polynomial feasibility system of Section 6, and solve it with a certified backend that captures isolated and positive-dimensional solutions.* This is correct (it cannot miss within its declared regime) and is validated on a known reduced game. The decisions still to be made, and to be settled by measurement before committing months of compute, are:

1. the per-label realisable-active-set count r and solve cost c with a fast backend (this fixes the total cost of Equation (3));
2. whether acceptance-only certification already resolves the running instance (if it certifies *no* acceptance-only equilibrium, that is itself the theorem “`burke_usaruschn` requires proposal mixing”);
3. the choice of certified backend (`msolve` vs. certified homotopy) and the distributed harness across the available cores.

A Peripheral sub-analyses (not load-bearing)

These items are not essential to the argument — the headline costs of Section 7 do not rest on them — but they are the hard numbers behind some statements, recorded for completeness.

A.1 A loose upper bound: “conceivable combinations”

One can bound the search by treating each cell’s proposal support as a free, independent choice. With supports $(15, 7, 7, 7, 15)$ per proposer, a full *support profile* (one choice for all 15 cells) numbers $15 \cdot 7 \cdot 7 \cdot 7 \cdot 15 = 77,175$ per proposer, so $77,175^3 \approx 4.6 \times 10^{14}$ profiles, and $\times 541^3 \approx 7.3 \times 10^{22}$ “labels”; restricting to pure proposals still leaves $432^3 \times 541^3 \approx 1.3 \times 10^{16}$.

This arithmetic is correct as a count of *conceivable* combinations, but it is *not* the work and is not a factor in any total. Proposals are not free: given a triple and the acceptance mixing, the proposer’s EV-optimal target is determined, so almost all 4.6×10^{14} profiles are immediately inconsistent and never formed. The work is governed instead by the *realisable* support count $r(R)$ of Section 7.4 (median 27), which is what enters the totals. We record 7.3×10^{22} only to retire it.

A.2 Variable elimination and realisable supports

Two reductions keep the per-label system small. (i) *Variable elimination*: a free acceptance variable on a transition no proposer ever proposes never enters P , hence not V , hence no equation — it is dropped; only acceptances on proposed transitions remain. (ii) *Few indifference equations*: their number is the total tied-group size minus the number of groups, summed over players — at most $S - 1 = 4$ per player (≤ 12 total), usually far fewer. Together these are why, despite up to ~ 35 raw mixing contexts, the effective system at a typical triple is tiny.

A.3 Summary of the key quantities

quantity	value	status
weak orders per player $a(5)$	541	exact
weak-order triples 541^3	1.58×10^8	exact (outer loop)
strict-order triples 120^3	1.73×10^6	exact (no-ties subset)
feasible targets / state	$(4, 3, 3, 3, 4)$	exact (adjacency; §7.4)
candidate supports / cell	$(15, 7, 7, 7, 15)$	exact
mixing dim $m(R)$	median 2, max 14	measured
solve cost $c(m)$	0.002 s $\rightarrow$ 18.6 s ($m:2 \rightarrow 8$)	measured; super-linear
support count $r(R)$	median 27, mean 1.3×10^6 , max 1.2×10^{11}	measured; heavy-tailed
$\rho_{\text{Spearman}}(r, m)$	0.44	measured
acceptance total (14w)	~ 27 days	measured, stable (6.5%)
full total (14w)	$\sim 6 \times 10^9$ days	Σrc ; unstable, infeasible